
Simulador de Gestor de Procesos para Sistemas Operativos

Materia	Sistemas Operativos
Institución	Universidad Autónoma de Tamaulipas
Semestre	Sexto Semestre
Profesor(es)	Dr. Muñoz Quintero Dante Adolfo
Integrantes	Arias Castillos Raul Aram, Atanasio Gómez Edson Felipe, Cabrera Martinez Jesus Alfonso, Gómez Juarez, Isaias
Repositorio	https://github.com/EdsonfAg/simulador-gestor-procesos.git
Fecha	17 de mayo de 2026

Sección 1

1. Introducción

El presente proyecto es un Simulador de Gestor de Procesos, fue desarrollado para la materia de Sistemas Operativos con el propósito de representar de forma visual el funcionamiento básico de la gestión de procesos. El sistema permite crear procesos simulados, mostrar sus estados, visualizar el uso de memoria, manejar colas de planificación y registrar eventos durante la ejecución.

El proyecto está desarrollado en C++ con interfaz gráfica en Qt6, lo que permite observar la información de manera más clara mediante tablas, botones, paneles y registros de actividad. Aunque no administra procesos reales del sistema operativo, sirve como una herramienta educativa para comprender cómo se organizan y controlan los procesos dentro de una simulación. El simulador ayuda a reforzar conceptos como creación de procesos, planificación, asignación de recursos, cambios de estado y seguimiento de eventos, facilitando el aprendizaje práctico de temas relacionados con los sistemas operativos.

Sección 2

2. Objetivos

- **El presente proyecto** es un **Simulador de Gestor de Procesos**, el funcionamiento básico de la administración de procesos en un sistema operativo, utilizando una interfaz gráfica en C++ con Qt.
- Crear procesos simulados con datos como nombre, PID, prioridad, ráfaga, memoria y estado.
- Mostrar los procesos en una interfaz gráfica clara mediante tablas, paneles y controles.
- Representar los cambios de estado de los procesos durante la simulación.
- Simular la asignación y liberación de recursos como memoria y CPU.
- Visualizar el uso de memoria del sistema durante la ejecución.
- Registrar los eventos principales del simulador para dar seguimiento a las acciones realizadas.
- Facilitar la comprensión de conceptos de sistemas operativos mediante una herramienta

Sección 3

3. Arquitectura del Sistema

3.1 InterfazGrafica

Elemento	Descripción
Clase	InterfazGrafica
Archivo principal	src/core/UI/InterfazGrafica.h / src/UI/InterfazGrafica.cpp
Tipo	Clase de interfaz gráfica; hereda de QMainWindow.
Propósito	Mostrar visualmente el estado del simulador y permitir la interacción del usuario con los procesos.
Atributo principal	MotorSimulacion& m_motor.
Componentes visuales	Listas de procesos listos y suspendidos, tabla de procesos, historial de logs, barra de memoria, botones, etiquetas y temporizador.
Métodos principales	construirInterfaz(), conectarSenales(), aplicarEstilo(), actualizarVistas(), mostrarColasPlanificacion(), listarProcesosYRecursos(), mostrarHistorialLogs(), capturarDatosProceso().
Relación con otras clases	Consulta y controla el simulador mediante MotorSimulacion.

3.2 MotorSimulacion

Elemento	Descripción
Clase	MotorSimulacion
Archivo principal	src/core/MotorSimulacion/MotorSimulacion.h / src/MotorSimulacion/MotorSimulacion.cpp
Tipo	Clase central de control.
Propósito	Coordinar el funcionamiento general del simulador.
Atributos principales	Planificador, GestorComunicacion, GestorRecursos, GestorLogs, ProductorConsumidor, siguiente_item y cola_nuevos.
Estructura auxiliar	Utiliza ProcesoPendiente para guardar procesos nuevos antes de cargarlos a memoria.
Métodos principales	iniciar(), crearProceso(), ejecutarPasoSiguiente(), intentarCargarProcesosAMemoria(), forzarSalidaProceso(), cambiarEstadoSuspension().
Relación con otras clases	Conecta la interfaz gráfica con planificación, recursos, comunicación, logs y productor-consumidor.

3.3 Planificador

Elemento	Descripción
Clase	Planificador
Archivo principal	src/core/Planificador/Planificador.h / src/planificador/Planificador.cpp
Tipo	Clase de administración y planificación de procesos.
Propósito	Controlar la tabla de procesos, las colas de planificación y la ejecución simulada de CPU.
Atributos principales	algoritmo, valor_quantum, ticks_ejecutados_quantum, reloj_global, tabla_procesos, cola_listos, cola_suspendidos, historial_terminados y pid_en_ejecucion.
Recurso interno	Contiene un objeto GestorRecursos.
Métodos principales	crearYAsignarProceso(), avanzarTiempo(), ejecutarCPU(), ejecutarCicloCompleto(), ejecutarDespachador(), suspender(), reanudar(), finalizar().
Algoritmos considerados	FCFS, SJF, ROUND_ROBIN y PRIORIDADES.
Relación con otras clases	Administra objetos Proceso y puede comunicarse con GestorComunicacion mediante llamadas al sistema.

3.4 Proceso

Elemento	Descripción
Clase	Proceso
Archivo principal	src/core/Planificador/Proceso.h / src/planificador/Proceso.cpp
Tipo	Clase modelo.
Propósito	Representar un proceso individual dentro del simulador.
Atributos principales	pid, nombre, estado, prioridad, rafaga_restante, memoria_asignada, recursos_fisicos, contador_programa y tipo_proceso.
Estados manejados	LISTO, EJECUTANDO, ESPERANDO y TERMINADO.
Tipos de proceso	NORMAL, PRODUCTOR y CONSUMIDOR.
Métodos principales	obtenerPid(), obtenerNombre(), obtenerEstado(), obtenerRafagaRestante(), obtenerPrioridad(), actualizarEstado(), actualizarMemoriaAsignada(), ejecutarUnTick().
Relación con otras clases	Es creado y administrado por Planificador.

3.5 GestorRecursos

Elemento	Descripción
Clase	GestorRecursos
Archivo principal	src/core/GestorRecursos/GestorRecursos.h / src/GestorRecursos/GestorRecursos.cpp
Tipo	Clase de administración de recursos.
Propósito	Controlar la memoria y CPU simuladas del sistema.
Atributos principales	max_memoria, max_cpus, memoria_usada, mapa_memoria, pids_con_cpu y logs.
Valores por defecto	Memoria máxima de 4096 MB y 1 CPU simulada.
Métodos de memoria	validarDisponibilidadMemoria(), asignarMemoria() y liberarMemoria().
Métodos de CPU	asignarCPU() y liberarCPU().
Getters principales	obtenerMemoriaUsada(), obtenerCPUsEnUso() y obtenerMemoriaMaxima().
Relación con otras clases	Es usado por MotorSimulacion y Planificador para validar recursos.

3.6 GestorComunicacion

Elemento	Descripción
Clase	GestorComunicacion
Archivo principal	src/core/GestorComunicacion/GestorComunicacion.h / src/GestorComunicacion/GestorComunicacion.cpp
Tipo	Clase de comunicación y sincronización.
Propósito	Simular mecanismos de comunicación entre procesos mediante semáforos.
Atributos principales	semaforos y procesos_bloqueados.
Métodos principales	inicializarSemaforo(), esperarSemaforo() y liberarSemaforo().
Funciones relacionadas	simularProductor() y simularConsumidor().
Relación con otras clases	Se relaciona con Planificador y ProductorConsumidor.

3.7 ProductorConsumidor

Elemento	Descripción
Clase	ProductorConsumidor
Archivo principal	src/core/ProductorConsumidor/ProductorConsumidor.h / src/ProductorConsumidor/ProductorConsumidor.cpp
Tipo	Clase de simulación de concurrencia.
Propósito	Representar el problema clásico productor-consumidor.
Atributos principales	buffer_compartido, CAPACIDAD_MAXIMA y seccion_critica_mtx.
Capacidad del buffer	5 elementos.
Métodos principales	simularProductor(), simularConsumidor(), obtenerTamanoBuffer() y obtenerCapacidadMaxima().
Relación con otras clases	Usa Planificador, GestorComunicacion y GestorLogs.

3.8 GestorLogs

Elemento	Descripción
Clase	GestorLogs
Archivo principal	src/core/GestorLogs/GestorLogs.h / src/GestorLogs/GestorLogs.cpp
Tipo	Clase de registro de eventos.
Propósito	Guardar eventos importantes generados durante la simulación.
Atributos principales	logsRAM, logsCPU e historialLogs.
Métodos generales	anotarEvento() y exportarHistorialLogs().
Métodos de RAM	logValidarDisponibilidadMemoria(), logAsignarMemoria(), logLiberarMemoria() y logInsuficienteMemoria().
Métodos de CPU	logAsignarCPU(), logLiberarCPU() y exportarLogsCPU().
Relación con otras clases	Es usado por GestorRecursos, MotorSimulacion y la interfaz gráfica.

3.9 main

Elemento	Descripción
Archivo	src/main.cpp
Tipo	Punto de arranque del programa.
Propósito	Inicializar Qt, crear el motor de simulación y mostrar la interfaz gráfica.
Componentes creados	QApplication app, MotorSimulacion motor e InterfazGrafica ventana.
Configuración inicial	Define nombre de aplicación, nombre visible, organización y estilo Fusion.
Relación con otras clases	Conecta MotorSimulacion con InterfazGrafica.

Sección 4

4. Diagramas UML y de Módulos del Sistema

4.1 Diagrama general de módulos / componentes

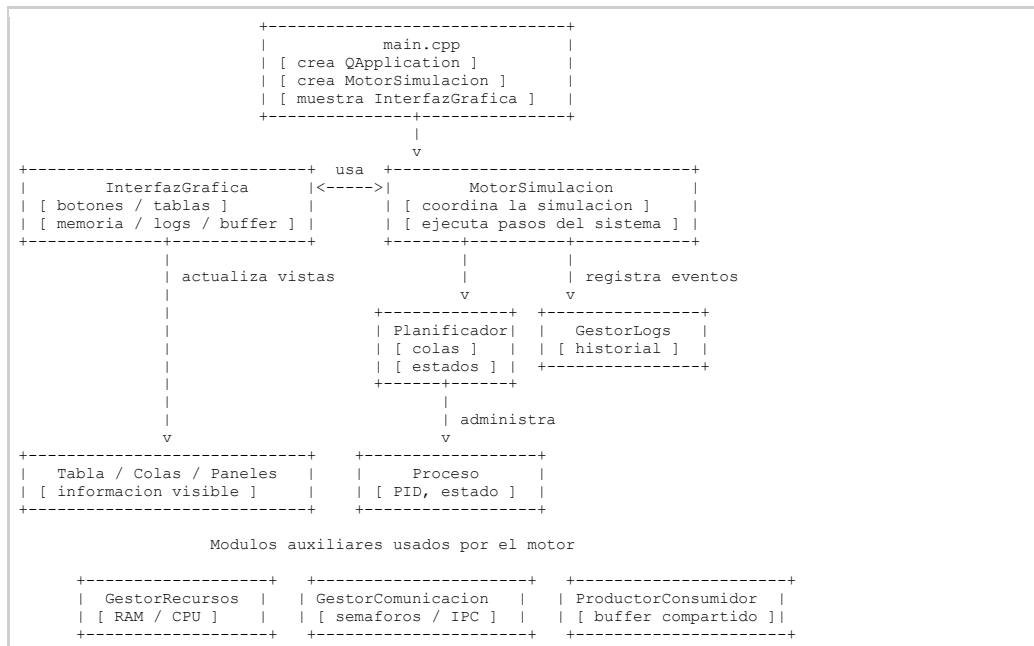


Figura 1. Diagrama general de módulos / componentes.

4.2 Flujo de datos entre módulos

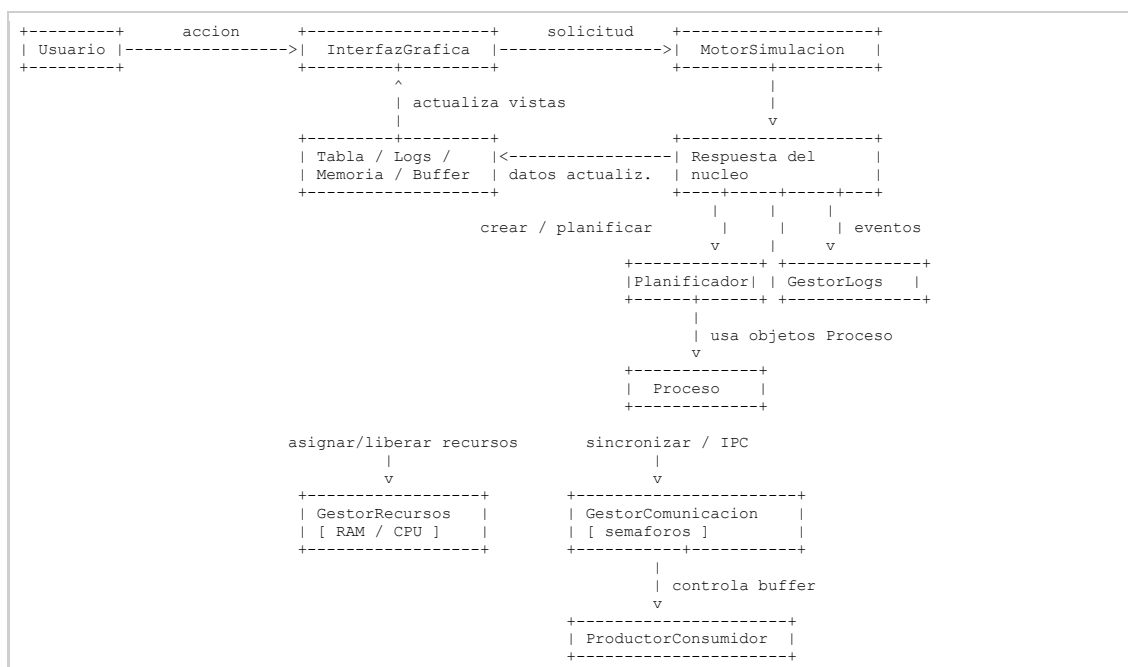


Figura 2. Flujo de datos entre módulos.

4.3 Diagrama de componentes

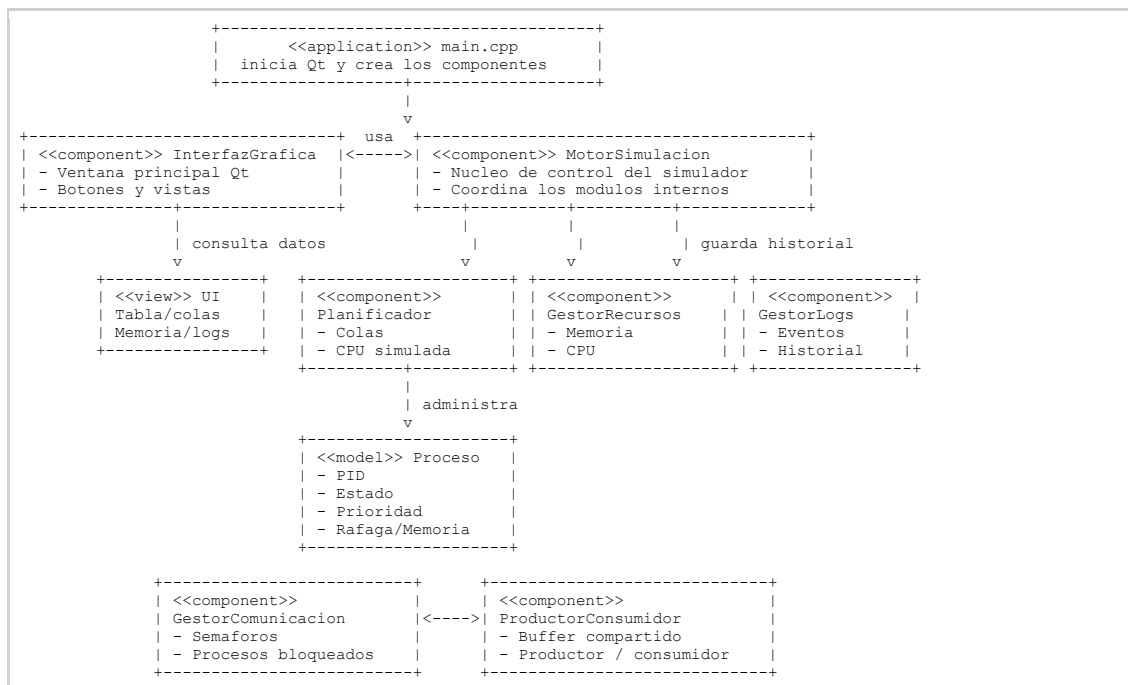


Figura 3. Diagrama de componentes.

4.4 Diagrama de estados del proceso

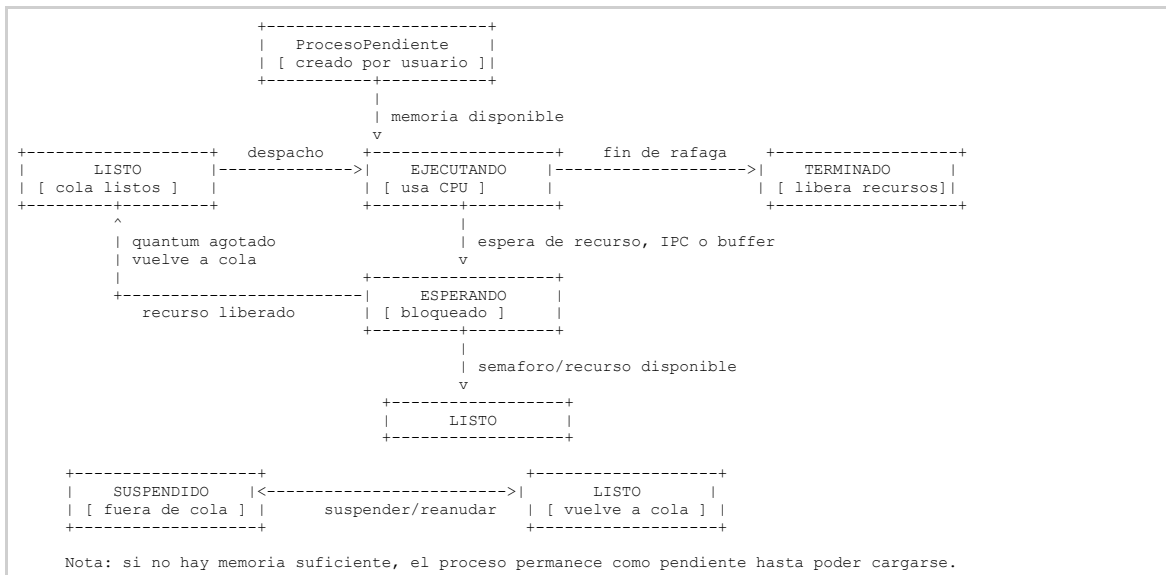


Figura 4. Diagrama de estados del proceso.

4.5 Diagrama UML del Simulador

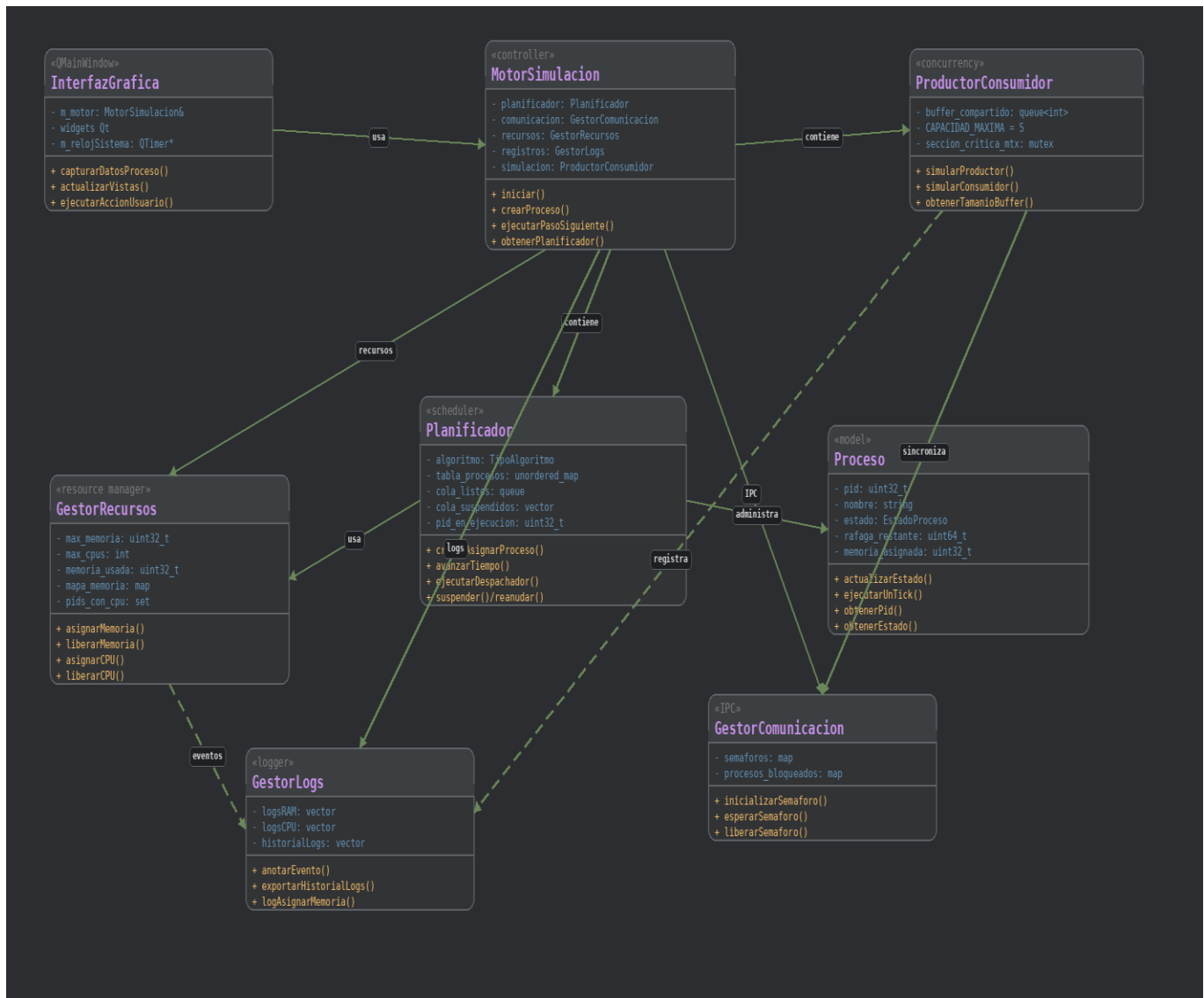


Figura 5. Diagrama general de clases.

4.6 Diagrama de clases del Núcleo

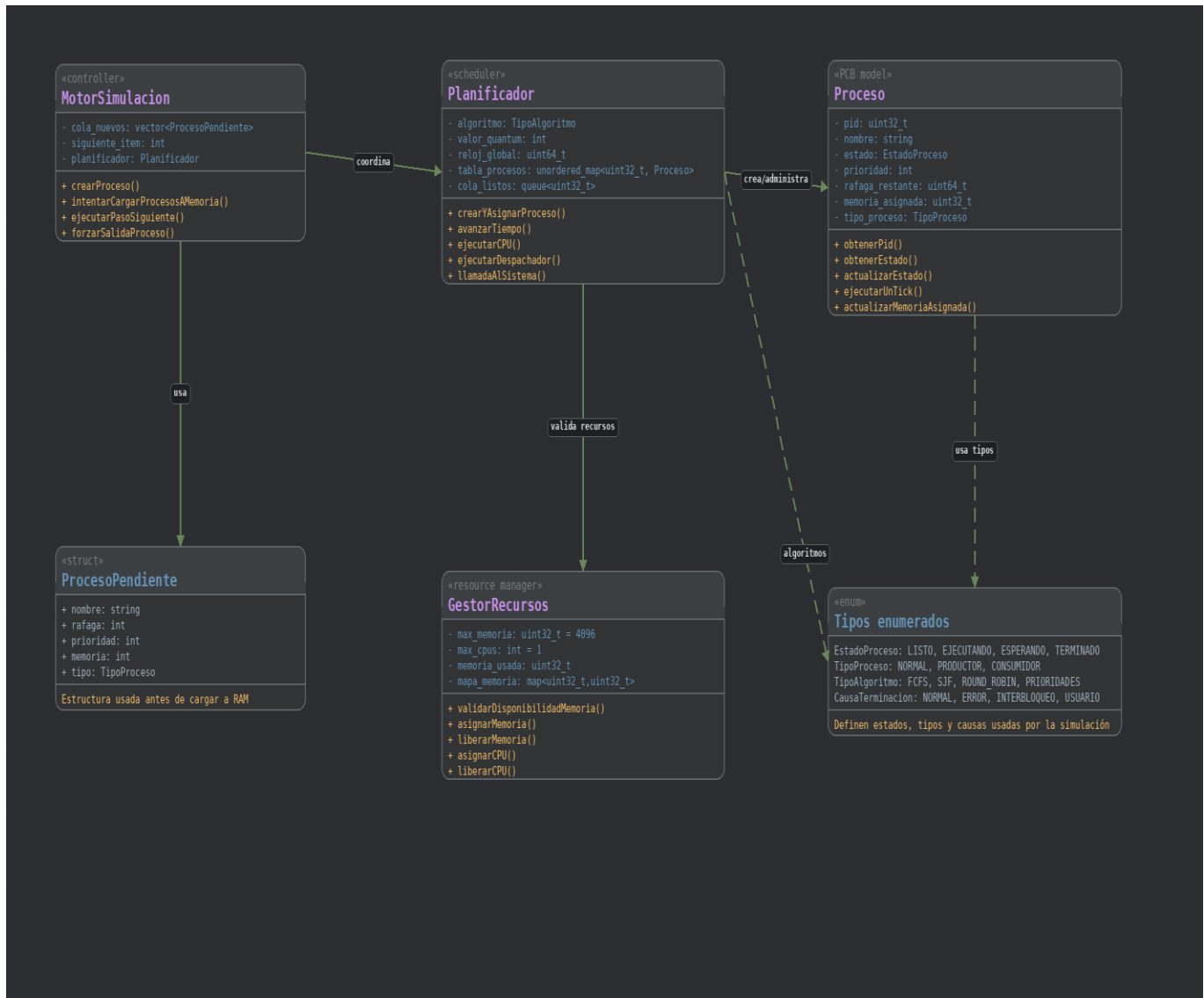


Figura 6. Diagrama de clases del núcleo.

4.7 Diagrama de clases de interfaz, Comunicación y Logs

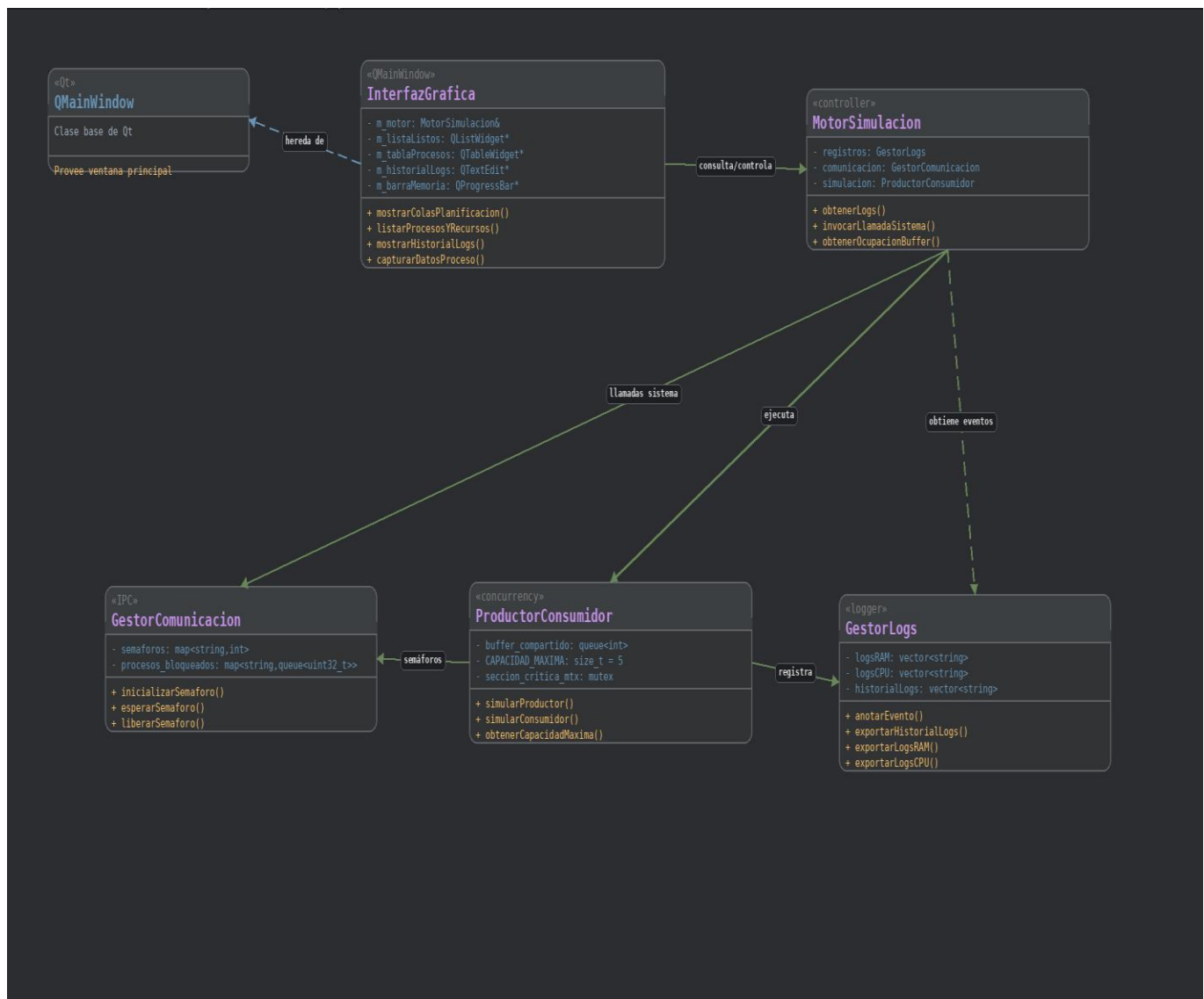


Figura 7. Diagrama de clases de interfaz, Comunicación y Logs.

Sección 5

5. Implementación

5.1 Punto de entrada — main.cpp

Este fragmento representa el arranque del programa. Primero se inicializa la aplicación Qt mediante QApplication, después se crea el objeto MotorSimulacion, que funciona como núcleo lógico del sistema, y finalmente se crea la ventana principal InterfazGrafica, conectándola con el motor de simulación. El método app.exec() mantiene activo el ciclo principal de eventos de la aplicación gráfica.

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    app.setApplicationName(QStringLiteral("SimuladorGestorProcesos"));
    app.setApplicationDisplayName(QStringLiteral("Simulador de Gestión de Procesos"));
    app.setOrganizationName(QStringLiteral("CLionProjects"));
    app.setStyle(QStyleFactory::create(QStringLiteral("Fusion")));

    MotorSimulacion motor;
    InterfazGrafica ventana(motor);

    ventana.show();

    return app.exec();
}
```

5.2 Interfaz gráfica — InterfazGrafica

La clase InterfazGrafica se encarga de construir y controlar la parte visual del simulador. Hereda de QMainWindow, por lo que funciona como ventana principal. Su atributo más importante es m_motor, ya que mediante él consulta el estado de los procesos, memoria, buffer y logs. También contiene métodos para mostrar colas de planificación, listar procesos, actualizar la vista y ejecutar acciones.

```
class InterfazGrafica final : public QMainWindow
{
    Q_OBJECT

public:
    explicit InterfazGrafica(MotorSimulacion& motor, QWidget *parent = nullptr);

    void mostrarColasPlanificacion();
    void listarProcesosYRecursos();
    void mostrarHistorialLogs();

public slots:
    void ejecutarAccionUsuario();
    void iniciarSimulacionAutomatica();

private:
    MotorSimulacion& m_motor;
    void construirInterfaz();
    void conectarSenales();
    void aplicarEstilo();
    void actualizarVistas();
};
```

5.3 Construcción de la interfaz

Este constructor recibe una referencia al motor de simulación y la guarda en `m_motor`. Después llama a los métodos encargados de construir la ventana, conectar botones y temporizadores, aplicar el estilo visual y cargar la primera actualización de datos. Esto permite separar la interfaz gráfica de la lógica interna del simulador.

```
InterfazGrafica::InterfazGrafica(MotorSimulacion &motor, QWidget *parent)
: QMainWindow(parent), m_motor(motor)
{
    construirInterfaz();
    conectarSenales();
    aplicarEstilo();
    actualizarVistas();
}
```

5.4 Motor de simulación — MotorSimulacion

La clase `MotorSimulacion` funciona como el núcleo del proyecto. Contiene los módulos principales del sistema: planificador, gestor de comunicación, gestor de recursos, gestor de logs y productor-consumidor. También maneja una cola de procesos nuevos que esperan ser cargados a memoria. Su función principal es coordinar el avance de la simulación y comunicar la interfaz gráfica con la lógica interna.

```
class MotorSimulacion
{
private:
    Planificador planificador;
    GestorComunicacion comunicacion;
    GestorRecursos recursos;
    GestorLogs registros;
    ProductorConsumidor simulacion;

    int siguiente_item;
    std::vector<ProcesoPendiente> cola_nuevos;

public:
    MotorSimulacion();

    bool crearProceso(std::string nombre, int rafaga, int prioridad,
        int memoria, TipoProceso tipo = TipoProceso::NORMAL);

    void ejecutarPasoSiguiente();
    void intentarCargarProcesosAMemoria();
};
```

5.5 Creación de procesos

Este método crea un proceso nuevo dentro del simulador. Primero valida si existe memoria suficiente para cargarlo. Si no hay memoria disponible, se registra el evento en los logs. Si la memoria es suficiente, el proceso se envía al planificador y después se le asigna memoria mediante el gestor de recursos.

```
bool MotorSimulacion::crearProceso(std::string nombre, int rafaga,
                                   int prioridad, int memoria, TipoProceso tipo)
{
    uint32_t memReq = static_cast<uint32_t>(memoria);

    const uint32_t memoriaDisponible =
        recursos.obtenerMemoriaMaxima() - recursos.obtenerMemoriaUsada();

    if (memReq > memoriaDisponible) {
        registros.logInsuficienteMemoria(memReq, memoriaDisponible);
        return false;
    }

    uint32_t pid = planificador.crearYAsignarProceso(
        nombre, prioridad, rafaga, memReq, tipo
    );

    if (!asignarMemoriaProceso(pid, memReq)) {
        return false;
    }

    return true;
}
```

5.6 Ejecución de un paso de simulación

Este fragmento muestra el ciclo principal de la simulación. En cada paso se intenta cargar procesos nuevos a memoria, se ejecuta el despachador del planificador, se revisa si el proceso actual es productor o consumidor, se simula su comportamiento correspondiente y finalmente se ejecuta un tick de CPU.

```
void MotorSimulacion::ejecutarPasoSiguiente()
{
    intentarCargarProcesosAMemoria();

    planificador.ejecutarDespachador();
    uint32_t pidActual = planificador.obtenerPidEnEjecucion();
    if (pidActual != 0) {
        const Proceso& proceso =
            planificador.obtenerDetallesProceso(pidActual);
        if (proceso.obtenerTipoProceso() == TipoProceso::PRODUCTOR) {
            simulacion.simularProductor(
                planificador, comunicacion, registros, pidActual, siguiente_item++
            );
        }
        else if (proceso.obtenerTipoProceso() == TipoProceso::CONSUMIDOR) {
            simulacion.simularConsumidor(
                planificador, comunicacion, registros, pidActual
            );
        }
    }

    planificador.ejecutarCPU();
    planificador.avanzarTiempo();
}
```

5.7 Proceso — Proceso

La clase Proceso representa un proceso individual dentro del simulador. Guarda información como PID, nombre, estado, prioridad, ráfaga restante, memoria asignada, contador de programa y tipo de proceso. Esta clase funciona como una representación básica del PCB, ya que almacena los datos necesarios para que el planificador pueda administrar cada proceso.

```
enum class EstadoProceso {
    LISTO,
    EJECUTANDO,
    ESPERANDO,
    TERMINADO
};
enum class TipoProceso {
    NORMAL,
    PRODUCTOR,
    CONSUMIDOR
};
class Proceso
{
private:
    uint32_t pid;
    std::string nombre;
    EstadoProceso estado;
    int prioridad;
    uint64_t rafaga_restante;
    uint32_t memoria_asignada;
    int contador_programa;
    TipoProceso tipo_proceso;

public:
    void ejecutarUnTick();
    bool actualizarEstado(EstadoProceso nuevo);
};
```

5.8 Planificador — Planificador

El Planificador administra la tabla de procesos, la cola de listos, la cola de suspendidos, el historial de procesos terminados y el proceso que está en ejecución. También guarda el algoritmo de planificación seleccionado y el quantum utilizado en Round Robin. Esta clase representa la parte encargada de decidir qué proceso pasa a CPU y cómo cambia su estado durante la simulación.

```
class Planificador
{
private:
    TipoAlgoritmo algoritmo;
    int valor_quantum;
    int ticks_ejecutados_quantum;
    uint64_t reloj_global;

    std::unordered_map<uint32_t, Proceso> tabla_procesos;
    std::queue<uint32_t> cola_listos;
    std::vector<uint32_t> cola_suspendidos;
    std::vector<uint32_t> historial_terminados;
    uint32_t pid_en_ejecucion;

public:
    uint32_t crearYAsignarProceso(std::string nombre, int prioridad,
                                   uint64_t rafaga, uint32_t memoria,
                                   TipoProceso tipo = TipoProceso::NORMAL);

    void ejecutarDespachador();
    void ejecutarCPU();
    void avanzarTiempo();
};
```

5.9 Creación y registro en el planificador

Este método crea un objeto `Proceso`, obtiene su PID, lo guarda dentro de la tabla central de procesos y después coloca su PID en la cola de listos. Con esto, el proceso queda preparado para competir por la CPU cuando el despachador lo seleccione.

```
uint32_t Planificador::crearYAsignarProceso(std::string nombre,
                                           int prioridad,
                                           uint64_t rafaga,
                                           uint32_t memoria,
                                           TipoProceso tipo)
{
    Proceso nuevo_proceso(nombre, prioridad, rafaga, memoria, tipo);
    uint32_t nuevo_pid = nuevo_proceso.obtenerPid();
    tabla_procesos.emplace(nuevo_pid, nuevo_proceso);
    cola_listos.push(nuevo_pid);
    return nuevo_pid;
}
```

5.10 Ejecución de CPU

Este método simula el trabajo del procesador. Si existe un proceso en ejecución, se le descuenta un tick de su ráfaga. También se incrementa el contador del quantum cuando se usa Round Robin. Si la ráfaga restante llega a cero, el proceso cambia a estado terminado y se libera la CPU.

```
void Planificador::ejecutarCPU()
{
    if (pid_en_ejecucion != 0) {
        Proceso& proceso_actual = tabla_procesos.at(pid_en_ejecucion);
        proceso_actual.ejecutarUnTick();
        ticks_ejecutados_quantum++;
        if (proceso_actual.obtenerRafagaRestante() == 0) {
            proceso_actual.actualizarEstado(EstadoProceso::TERMINADO);
            historial_terminados.push_back(pid_en_ejecucion);
            pid_en_ejecucion = 0;
        }
    }
}
```

5.11 Gestión de recursos — GestorRecursos

El `GestorRecursos` controla los recursos simulados del sistema, principalmente memoria RAM y CPU. Guarda la memoria máxima, la memoria utilizada, los procesos con memoria asignada y los procesos que tienen CPU. También puede conectarse con el gestor de logs para registrar eventos relacionados con la asignación o liberación de recursos.

```
class GestorRecursos
{
private:
    uint32_t max_memoria;
    int max_cpus;
    uint32_t memoria_usada;
    std::map<uint32_t, uint32_t> mapa_memoria;
    std::set<uint32_t> pids_con_cpu;
    GestorLogs* logs;
public:
    bool validarDisponibilidadMemoria(uint32_t mb) const;
    bool asignarMemoria(uint32_t pid, uint32_t mb);
    uint32_t liberarMemoria(uint32_t pid);

    bool asignarCPU(uint32_t pid);
    void liberarCPU(uint32_t pid);
};
```

5.12 Asignación y liberación de memoria

Estos métodos se encargan de asignar y liberar memoria. Antes de asignar memoria se valida si existe espacio disponible. Cuando un proceso recibe memoria, se actualiza el mapa de memoria y la cantidad usada. Al liberar memoria, se elimina el registro del proceso y se resta la memoria ocupada.

```
bool GestorRecursos::asignarMemoria(uint32_t pid, uint32_t mb)
{
    if (!validarDisponibilidadMemoria(mb)) {
        return false;
    }

    if (mapa_memoria.count(pid)) {
        memoria_usada -= mapa_memoria[pid];
    }

    memoria_usada += mb;
    mapa_memoria[pid] = mb;

    if (logs) {
        logs->logAsignarMemoria(pid, mb);
    }

    return true;
}

uint32_t GestorRecursos::liberarMemoria(uint32_t pid)
{
    auto it = mapa_memoria.find(pid);

    if (it != mapa_memoria.end()) {
        uint32_t mb_liberados = it->second;
        memoria_usada -= mb_liberados;
        mapa_memoria.erase(it);

        if (logs) {
            logs->logLiberarMemoria(pid, mb_liberados);
        }
        return mb_liberados;
    }
    return 0;
}
```

5.13 Comunicación y sincronización — GestorComunicacion

El GestorComunicacion simula mecanismos de sincronización mediante semáforos. Guarda los permisos disponibles de cada recurso y una cola de procesos bloqueados. Esto permite representar situaciones donde un proceso debe esperar hasta que un recurso compartido esté disponible.

```
class GestorComunicacion
{
private:
    std::map<std::string, int> semaforos;
    std::map<std::string, std::queue<uint32_t>> procesos_bloqueados;
public:
    void inicializarSemaforo(std::string nombre_recurso,
        int permisos_iniciales);
    bool esperarSemaforo(std::string nombre_recurso, uint32_t pid);

    uint32_t liberarSemaforo(std::string nombre_recurso);
};
```


5.14 Operaciones WAIT y SIGNAL

La operación `esperarSemaforo` representa el comportamiento de WAIT. Si el recurso tiene permisos disponibles, el proceso continúa. Si no hay permisos, el proceso se agrega a la cola de bloqueados. La operación `liberarSemaforo` representa SIGNAL; aumenta el permiso del recurso y despierta a un proceso bloqueado si existe alguno esperando.

```
bool GestorComunicacion::esperarSemaforo(std::string nombre_recurso,
                                         uint32_t pid)
{
    if (semaforos[nombre_recurso] > 0) {
        semaforos[nombre_recurso]--;
        return true;
    }

    procesos_bloqueados[nombre_recurso].push(pid);
    return false;
}

uint32_t GestorComunicacion::liberarSemaforo(std::string nombre_recurso)
{
    semaforos[nombre_recurso]++;

    if (!procesos_bloqueados[nombre_recurso].empty()) {
        uint32_t pid_despertado =
            procesos_bloqueados[nombre_recurso].front();

        procesos_bloqueados[nombre_recurso].pop();
        return pid_despertado;
    }

    return 0;
}
```

5.15 Productor-consumidor — ProductorConsumidor

La clase `ProductorConsumidor` representa el problema clásico de concurrencia donde varios procesos producen y consumen elementos de un buffer compartido. El buffer tiene una capacidad máxima de 5 elementos y se utiliza un mutex para proteger la sección crítica.

```
class ProductorConsumidor
{
private:
    std::queue<int> buffer_compartido;
    const size_t CAPACIDAD_MAXIMA = 5;
    std::mutex seccion_critica_mtx;

public:
    void simularProductor(Planificador& planificador,
                          GestorComunicacion& ipc,
                          GestorLogs& logs,
                          uint32_t pid,
                          int item);

    void simularConsumidor(Planificador& planificador,
                           GestorComunicacion& ipc,
                           GestorLogs& logs,
                           uint32_t pid);
};
```

5.16 Producción y consumo de elementos

Este fragmento muestra una parte de la simulación del productor. Primero se valida si existen espacios vacíos en el buffer. Si no hay espacio, el proceso se bloquea. Si puede continuar, entra a la sección crítica, agrega un elemento al buffer y después libera un permiso indicando que ahora existe un ítem disponible para los consumidores.

```
void ProductorConsumidor::simularProductor(Planificador& planificador,
                                           GestorComunicacion& ipc,
                                           GestorLogs& logs,
                                           uint32_t pid,
                                           int item)
{
    if (!planificador.LlamadaAlSistema(
        TipoLlamada::WAIT_SEMAFORO,
        "Espacios_Vacios",
        ipc)) {
        logs.anotarEvento("Productor bloqueado: buffer lleno.");
        return;
    }

    std::lock_guard<std::mutex> lock(seccion_critica_mtx);

    if (buffer_compartido.size() < CAPACIDAD_MAXIMA) {
        buffer_compartido.push(item);
        logs.anotarEvento("Ítem producido correctamente.");
    }

    planificador.LlamadaAlSistema(
        TipoLlamada::SIGNAL_SEMAFORO,
        "Ítems_Disponibles",
        ipc);
}
```

5.17 Registro de eventos — GestorLogs

La clase GestorLogs guarda el historial de eventos del simulador. Separa eventos relacionados con RAM, CPU y un historial general. Esto permite que la interfaz gráfica pueda mostrar lo que ocurre durante la ejecución del sistema.

```
class GestorLogs
{
private:
    std::vector<std::string> logsRAM;
    std::vector<std::string> logsCPU;
    std::vector<std::string> historialLogs;

public:
    void anotarEvento(const std::string& mensaje);

    std::vector<std::string> exportarHistorialLogs() const;

    void logAsignarMemoria(uint32_t pid, uint32_t mb);
    void logLiberarMemoria(uint32_t pid, uint32_t mb);
    void logInsuficienteMemoria(uint32_t requerido, uint32_t disponible);

    void logAsignarCPU(uint32_t pid);
    void logLiberarCPU();
};
```

5.18 Escritura de logs

Estos métodos permiten guardar eventos dentro del historial del sistema. `anotarEvento` agrega cualquier mensaje al historial general y lo imprime en consola. `logAsignarMemoria` crea un mensaje específico cuando un proceso recibe memoria y después lo registra en los logs.

```
void GestorLogs::anotarEvento(const std::string& mensaje)
{
    historialLogs.push_back(mensaje);
    std::cout << mensaje << std::endl;
}

void GestorLogs::logAsignarMemoria(uint32_t pid, uint32_t mb)
{
    std::string mensaje =
        "PID " + std::to_string(pid) +
        " asigna " + std::to_string(mb) + " MB";
    logsRAM.push_back(mensaje);
    anotarEvento(mensaje);
}
```

5.19 Actualización de vistas en la interfaz gráfica

Este método actualiza la información visible en la interfaz. Refresca las colas de planificación, la tabla de procesos, el historial de logs, la barra de memoria, el estado del buffer compartido y la cantidad de procesos pendientes de memoria.

```
void InterfazGrafica::actualizarVistas()
{
    mostrarColasPlanificacion();
    listarProcesosYRecursos();
    mostrarHistorialLogs();
    const uint32_t usados = m_motor.obtenerRecursos().obtenerMemoriaUsada();
    m_barraMemoria->setValue(usados);
    m_barraMemoria->setFormat(tr("%v MB / %m MB usados"));
    const size_t ocupacion = m_motor.obtenerOcupacionBuffer();
    const size_t capacidad = m_motor.obtenerCapacidadBuffer();
    m_labelBuffer->setText(tr("Buffer: %1/%2").arg(ocupacion).arg(capacidad));
    const size_t enEspera = m_motor.obtenerProcesosNuevosPendientes();
    m_lblEsperandoMemoria->setText(
        tr("Esperando Memoria: %1 procesos").arg(enEspera)
    );
}
```

5.20 Suspensión y reanudación de procesos

Este fragmento representa el cambio de estado entre suspensión y reanudación. Cuando se suspende un proceso, se mueve a la cola correspondiente y se registra el evento. Cuando se reanuda, el proceso vuelve a la cola de listos para poder ser tomado nuevamente por el planificador.

```
void MotorSimulacion::cambiarEstadoSuspension(uint32_t pid, bool suspender)
{
    if (suspender) {
        planificador.suspender(pid);
        registros.anotarEvento(
            "PID " + std::to_string(pid) + " enviado a cola de suspendidos."
        );
    }
    else {
        planificador.reanudar(pid);
        registros.anotarEvento(
            "PID " + std::to_string(pid) + " regresado a cola de listos."
        );
    }
}
```

5.21 Configuración de compilación — CMakeLists.txt

Este archivo configura la compilación del proyecto. Define el estándar de C++, activa las herramientas automáticas de Qt para procesar archivos de interfaz y macros, busca el módulo Qt Widgets, recopila los archivos fuente dentro de src y crea el ejecutable SimuladorGestorProcesos. También enlaza el proyecto con Qt6::Widgets, lo que permite utilizar elementos como QApplication, QMainWindow, botones, tablas y demás componentes gráficos.

```
cmake_minimum_required(VERSION 3.16)
project(SimuladorGestorProcesos)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(CMAKE_AUTOUIC ON)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)

set(CMAKE_PREFIX_PATH "C:/Qt/6.11.1/mingw_64" ${CMAKE_PREFIX_PATH})

find_package(Qt6 COMPONENTS Widgets REQUIRED)

file(GLOB_RECURSE SOURCES
    "src/*.cpp"
    "src/*.h"
    "src/*.ui"
)

add_executable(SimuladorGestorProcesos ${SOURCES})

target_include_directories(SimuladorGestorProcesos PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/src
    ${CMAKE_CURRENT_BINARY_DIR}
)

target_link_libraries(SimuladorGestorProcesos PRIVATE Qt6::Widgets)
```

Sección 6

6. Demostración Funcional

6.1 Selección del algoritmo FCFS / FIFO

En este paso se selecciona el algoritmo de planificación que utilizará el simulador. Para esta demostración se eligió **FCFS / FIFO**, el cual atiende los procesos en el orden en que llegan a la cola de listos.

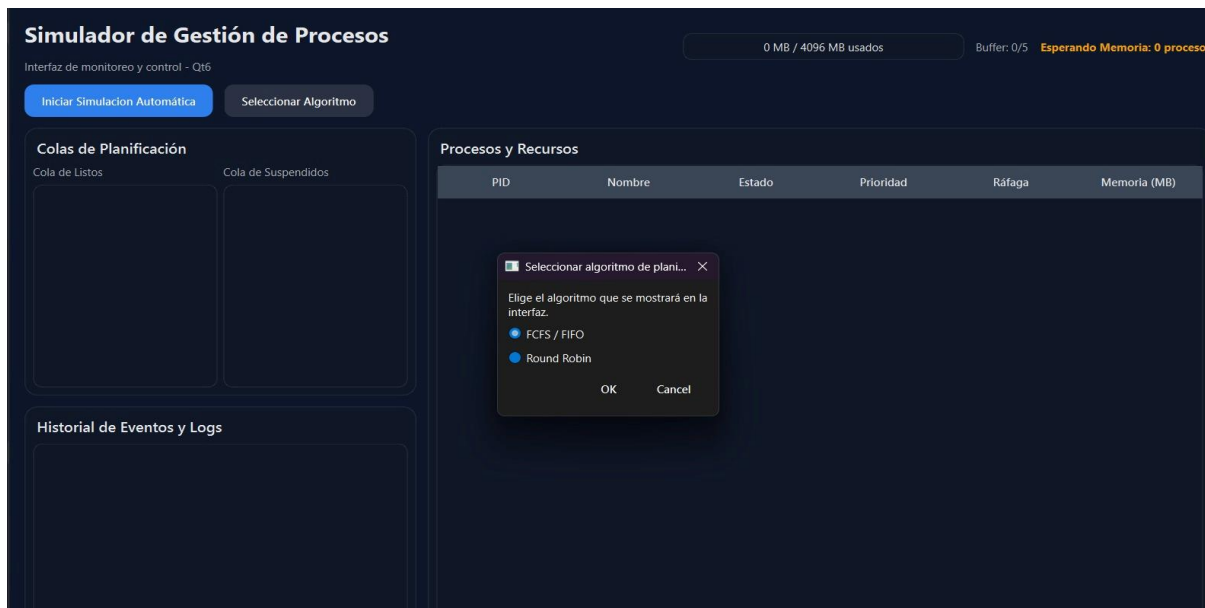


Figura 8. Selección del algoritmo FCFS / FIFO.

6.2 Configuración de la simulación automática

Después de elegir el algoritmo, se configura la simulación automática. En esta ventana se indica la cantidad de procesos que se van a generar y la velocidad del reloj o tick, permitiendo controlar cómo avanzará la ejecución.

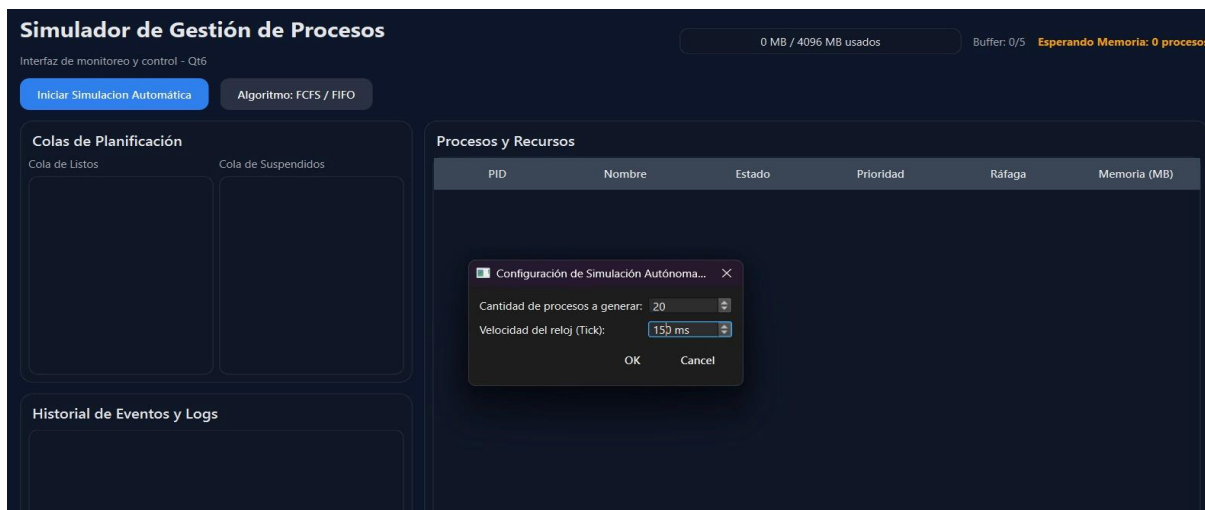


Figura 9. Configuración de la simulación automática para FCFS / FIFO.

6.3 Ejecución de la simulación

Al iniciar la simulación, los procesos comienzan a cargarse en memoria y se muestran en la tabla principal con datos como PID, nombre, estado, prioridad, ráfaga y memoria asignada. También se observa la cola de listos, el uso de memoria y el historial de eventos generados.

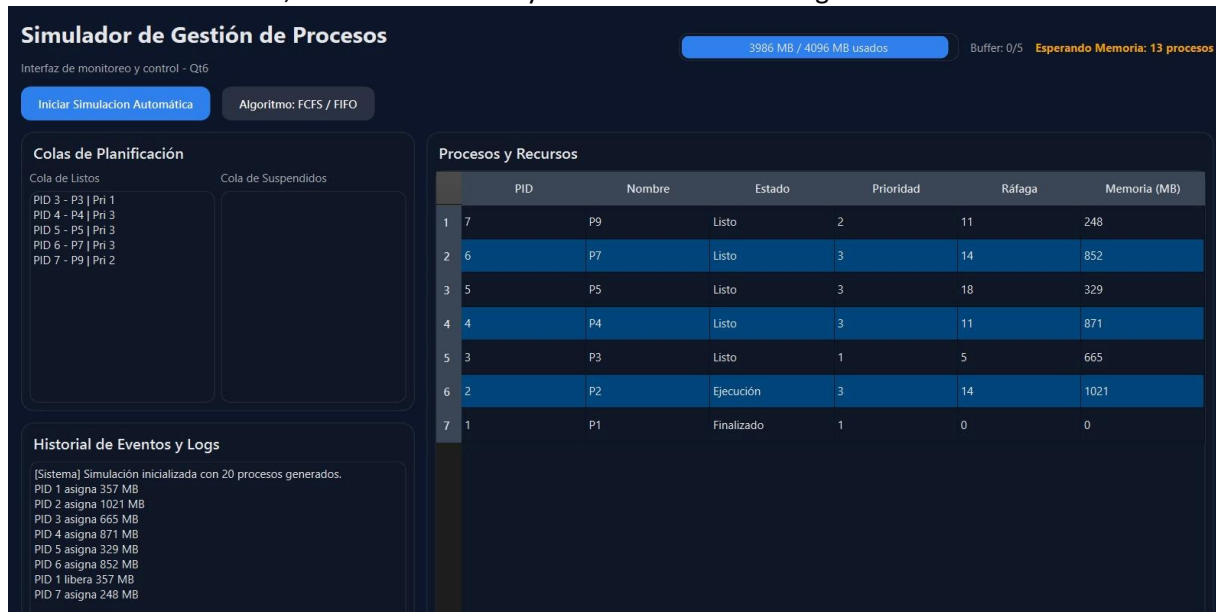


Figura 10. Inicio de la simulación con procesos cargados en FCFS / FIFO.

6.4 Finalización de procesos y liberación de recursos

Durante el avance de la simulación, los procesos van cambiando de estado hasta llegar a **Finalizado**. Cuando terminan, el sistema libera la memoria asignada y registra estas acciones en el historial de eventos, mostrando que los recursos fueron recuperados correctamente.

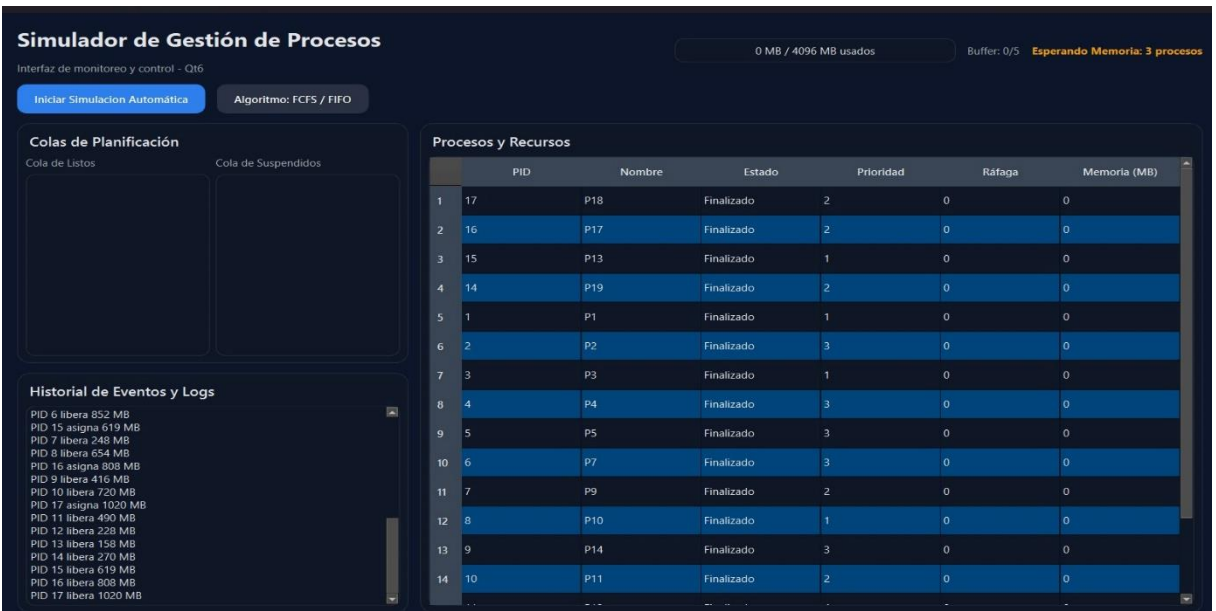


Figura 11. Finalización de procesos y liberación de recursos en FCFS / FIFO.

6.5 Selección del algoritmo Round Robin

En este paso se muestra la ventana de selección del algoritmo de planificación. Para esta segunda demostración se elige **Round Robin**, el cual permite ejecutar los procesos por turnos mediante un valor de quantum.

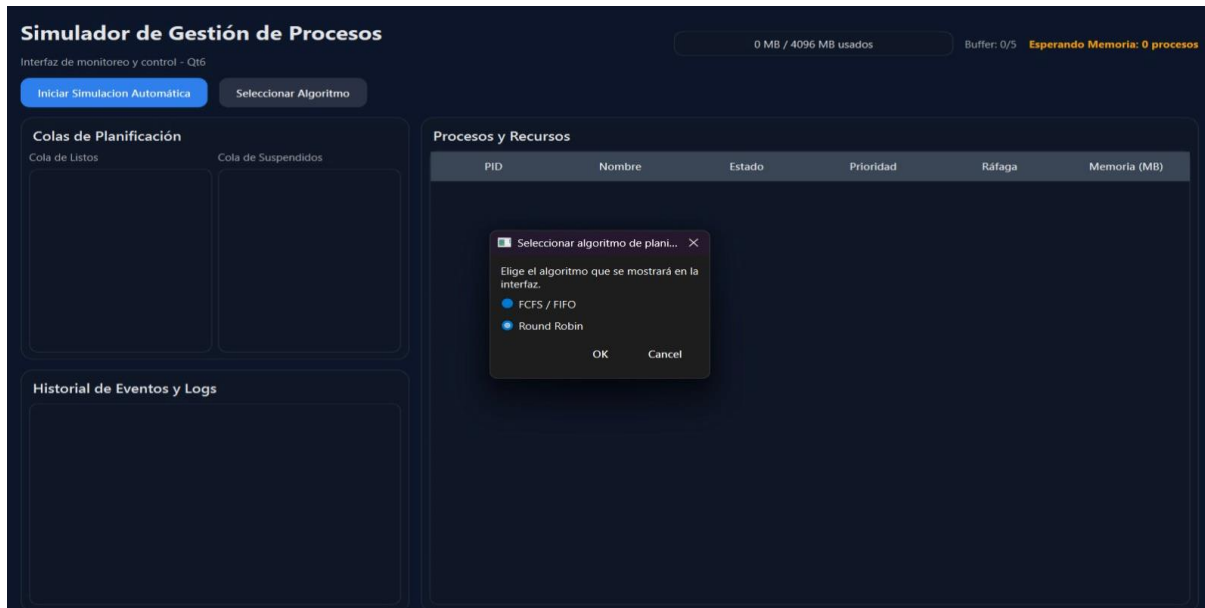


Figura 12. Selección del algoritmo Round Robin.

6.6 Configuración de la simulación automática

Después de seleccionar el algoritmo, se configura la simulación automática. En esta prueba se indica la cantidad de procesos que se van a generar y la velocidad del reloj, permitiendo controlar el avance de la simulación.

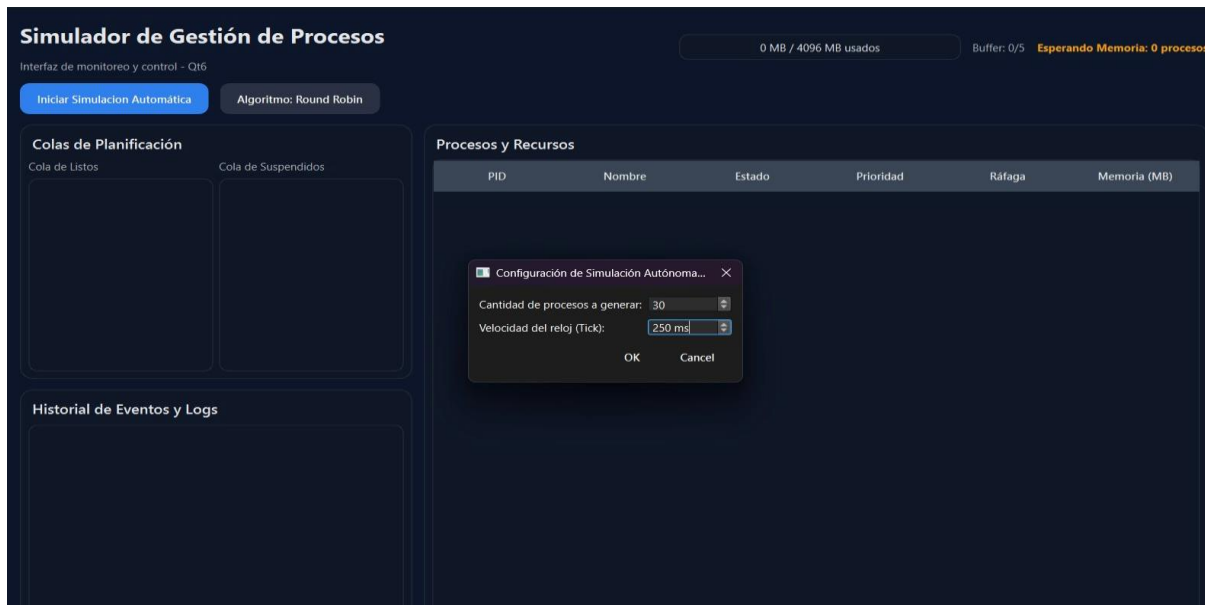


Figura 13. Configuración de la simulación automática para Round Robin.

6.7 Configuración del quantum

Antes de iniciar la ejecución con Round Robin, el sistema solicita el valor del **quantum**, que representa la cantidad de ticks que puede usar un proceso antes de ceder la CPU a otro proceso.

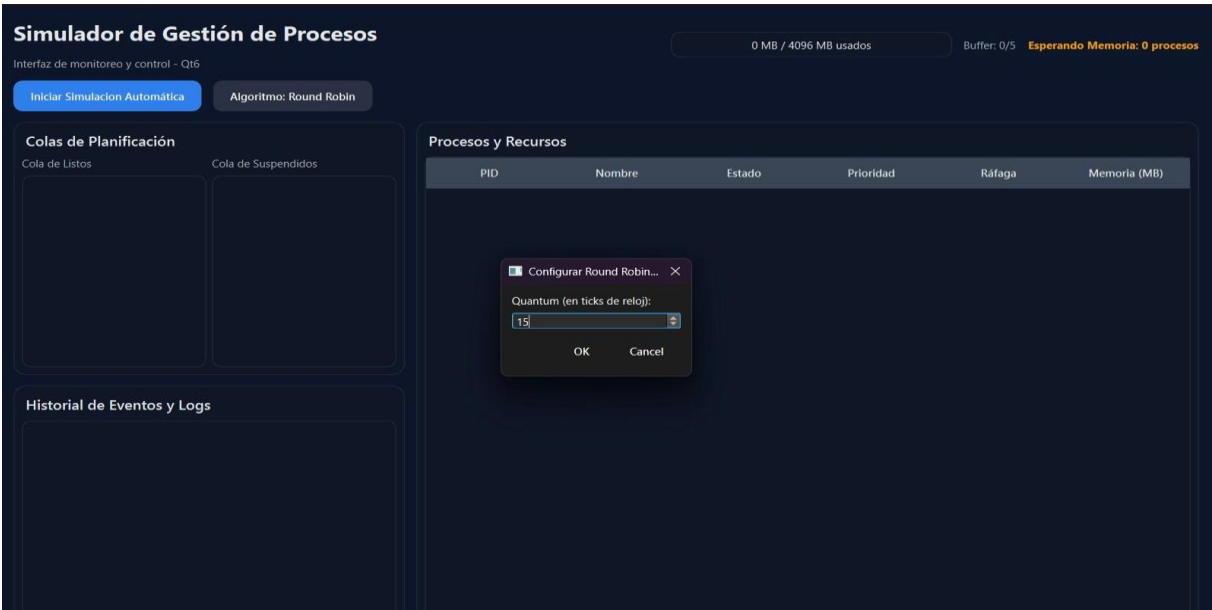


Figura 14. Configuración del quantum para Round Robin.

6.8 Inicio de la simulación con Round Robin

Al iniciar la simulación, los procesos comienzan a cargarse en memoria y se muestran en la tabla principal con sus datos correspondientes. También se observa la cola de listos, el proceso en ejecución, el uso de memoria y el historial de eventos.

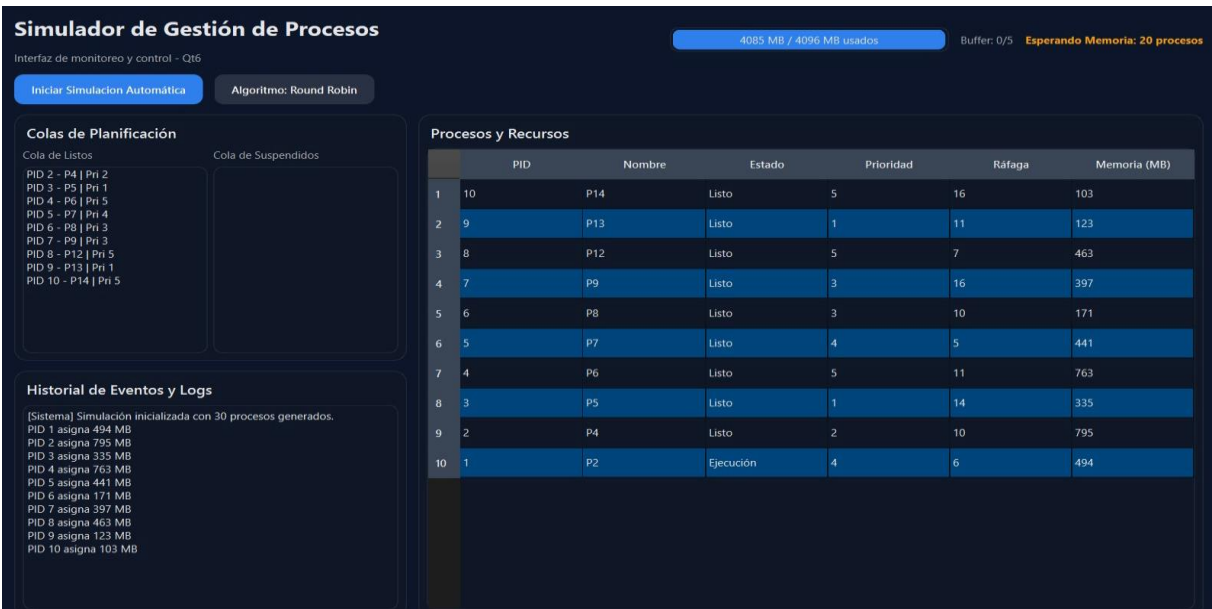


Figura 15. Inicio de la simulación con procesos cargados en Round Robin.

6.9 Ejecución, suspensión y productor-consumidor

Durante la ejecución se observa que algunos procesos cambian de estado, incluyendo procesos listos, finalizados, suspendidos y en ejecución. También se registra actividad relacionada con el productor-consumidor, donde el buffer puede llenarse y provocar bloqueos.

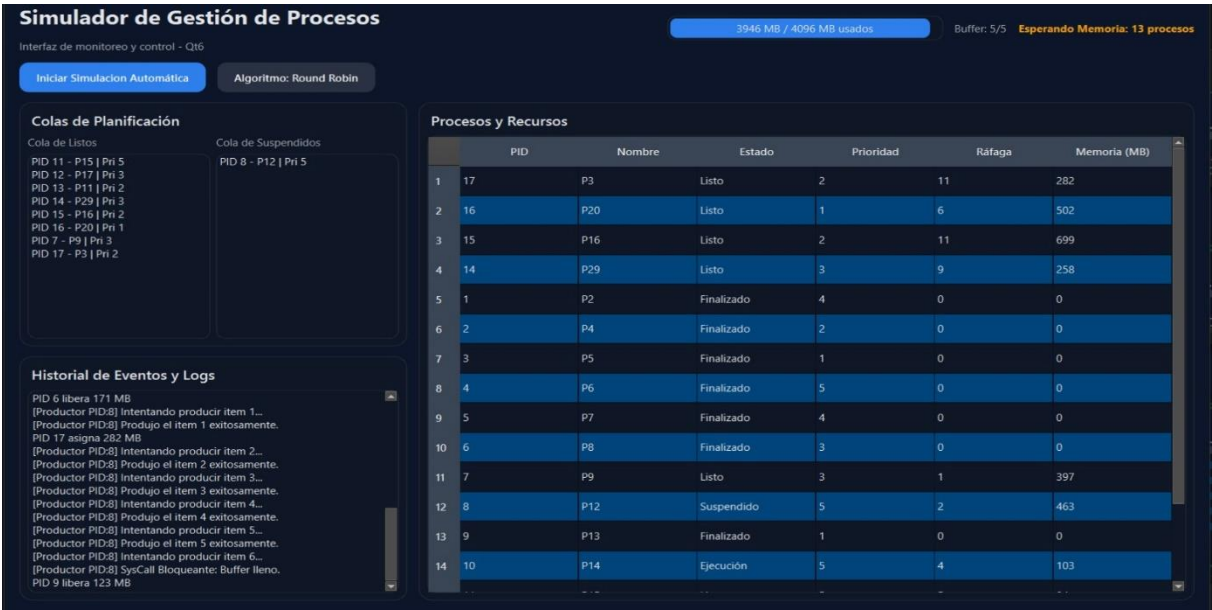


Figura 16. Ejecución de procesos, suspensión y eventos de productor-consumidor en Round Robin.

6.10 Finalización y liberación de recursos

En la etapa final, la mayoría de los procesos aparecen como **Finalizado**, mientras algunos quedan suspendidos o esperando memoria. El historial muestra la liberación de memoria y eventos de sincronización, demostrando que el simulador actualiza estados y recursos durante la ejecución.

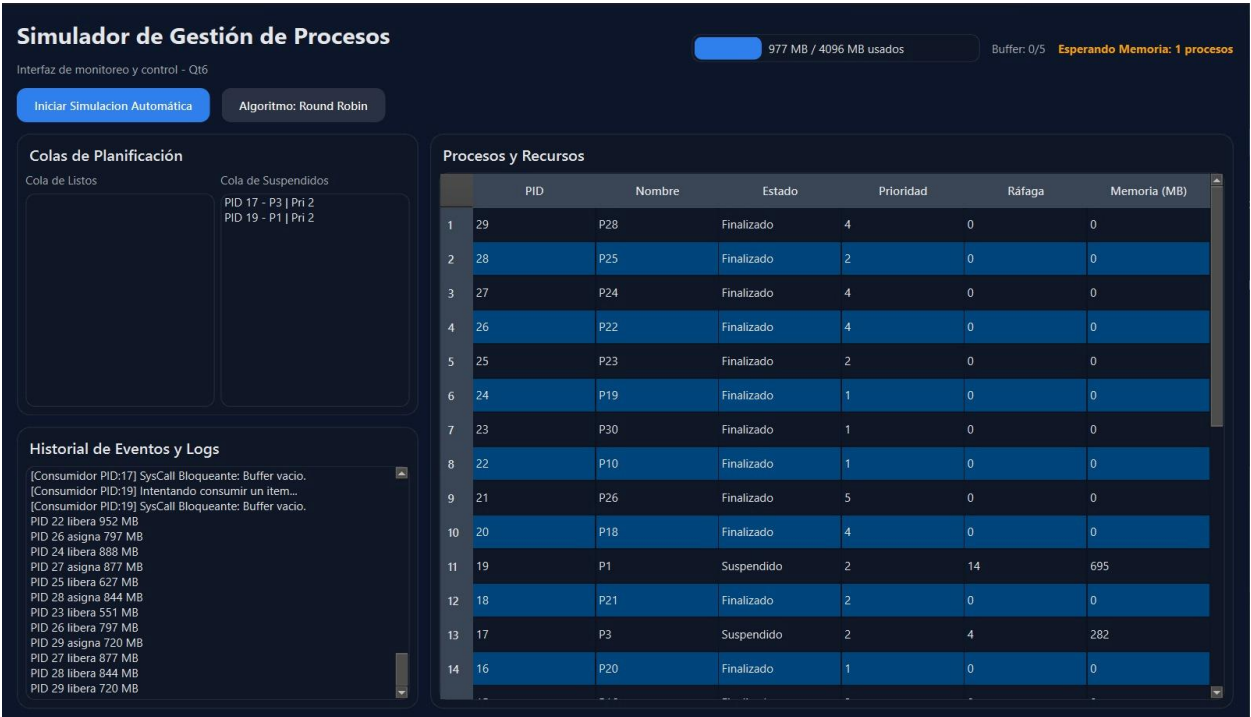


Figura 17. Finalización de procesos y liberación de recursos en Round Robin.

Sección 7

7. Conclusiones

Arias Castillos Raul Aram Durante el desarrollo del Simulador de Gestor de Procesos pude comprender mejor cómo se organizan y administran los procesos dentro de un sistema operativo. El proyecto permitió observar de forma práctica conceptos como la creación de procesos, los cambios de estado, la planificación y el uso de recursos. Además, trabajar con una interfaz gráfica ayudó a representar estos elementos de manera más clara y ordenada.

Atanasio Gómez Edson Felipe: La realización de este proyecto permitió aplicar conocimientos de Sistemas Operativos en una simulación funcional. A través del uso de C++ y Qt fue posible construir una herramienta visual que muestra el comportamiento de los procesos, la memoria simulada y las colas de planificación. Esto facilitó la comprensión de cómo interactúan los distintos componentes de un gestor de procesos.

Cabrera Martinez Jesus Alfonso: El proyecto ayudó a reforzar los temas vistos en clase, especialmente la gestión de procesos, la asignación de recursos y el registro de eventos. Aunque el simulador no trabaja con procesos reales del sistema operativo, cumple su objetivo educativo al mostrar de manera controlada cómo se comportan los procesos dentro de una simulación. También permitió comprender la importancia de separar la lógica del sistema y la interfaz gráfica.

Gómez Juarez Isaías: Desarrollar el simulador permitió observar de forma más práctica el funcionamiento básico de un gestor de procesos. La simulación muestra cómo los procesos pueden cambiar de estado, utilizar memoria, esperar recursos y ser controlados mediante una planificación. En general, el proyecto sirvió para relacionar la teoría de Sistemas Operativos con una implementación visual y funcional.

Sección 8

8. Referencias

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.
- Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.
- Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3rd ed.). Pearson.
- Nutt, G. J. (2004). *Operating systems: A modern perspective* (2nd ed.). Pearson.